

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт по лабораторной работе № 6

«62. Unique Paths»

Выполнила работу

Бездверная Полина Вячеславовна

Академическая группа J3112

Принято

Ментор, Дунаев Максим Владимирович

Санкт-Петербург

2025

1. Введение

В данной лабораторной работе была рассмотрена задача 62. Unique Paths, доступная на сайте [leetcode.com](https://leetcode.com/problems/unique-paths/) и имеющая уровень «medium». Условие задачи представлено на изображении 1.

62. Unique Paths

Medium Topics Companies

There is a robot on an $m \times n$ grid. The robot is initially located at the **top-left corner** (i.e., `grid[0][0]`). The robot tries to move to the **bottom-right corner** (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

Given the two integers `m` and `n`, return *the number of possible unique paths that the robot can take to reach the bottom-right corner*.

The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:



Input: `m = 3, n = 7`
Output: 28

Изображение 1 – Условие задачи

Цель: написать программу, которая для поля размера $M \times N$ будет находить количество возможных способов добраться из крайней левой клетки в крайнюю правую и проанализировать её работу.

Задачи:

1. Решить задачу с использованием динамического программирования
2. Проанализировать затрачиваемые время и память

3. Понять и объяснить, почему лучшее решение задачи – динамическое программирование

2. Теоретическая подготовка

В ходе выполнения данной работы для решения задачи был использован метод динамического программирования. Динамическое программирование (Dynamic Programming, ДП) — это способ решения сложных задач путём разбиения их на более простые подзадачи. Он применим к задачам с оптимальной подструктурой, выглядящим как набор перекрывающихся подзадач, сложность которых чуть меньше исходной. В этом случае время вычислений, по сравнению с «наивными» методами, можно значительно сократить.

Как правило, чтобы решить поставленную задачу, требуется решить отдельные подзадачи, после чего объединить их решения в одно общее. Часто многие из этих подзадач одинаковы. Подход динамического программирования состоит в том, чтобы решить каждую подзадачу только один раз, сократив тем самым количество вычислений, что особенно полезно в случаях, когда число повторяющихся подзадач экспоненциально велико.

Также были использованы следующие типы данных:

— `vector<int>` для хранения массивов

— `int` для хранения целых чисел

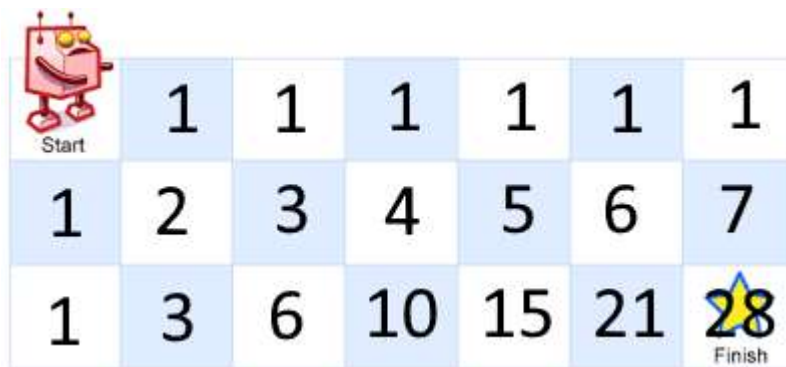
3. Реализация

3.1. Анализ задачи, поиск решения

В данной задаче требуется найти количество всевозможных способов, которыми робот может попасть из левой верхней клетки в правую нижнюю. При этом сам робот может двигаться только вправо и вниз. Оптимизация за счёт динамического программирования в данном случае достигается благодаря

сохранению промежуточных результатов в отдельном массиве, из-за чего не приходится делать много повторных вычислений для каждой клетки, а также не требуется рекурсивного подхода.

Стоит упомянуть, что обычно динамическое программирование требует дополнительные массивы, зачастую таблицы, для хранения промежуточных результатов, что может привести к большим затратам памяти. Например, полная таблица результатов для поля размером 3x7 будет выглядеть как представлено на изображении 2. (Исходное изображение, без цифр, взято со страницы задачи)



 Start	1	1	1	1	1	1
1	2	3	4	5	6	7
1	3	6	10	15	21	 Finish

Изображение 2 – Таблица промежуточных результатов

Однако, для вычисления количества способов попасть в определённую клетку в моменте нам достаточно знать только количество способов попасть в клетки, находящиеся в обратном направлении возможных передвижений робота, то есть слева и сверху от исходной. Поэтому в данном случае было принято решение хранить и использовать только два ряда из всей таблицы: рассматриваемый в данный момент и предыдущий.

3.2. Написание кода

На данном этапе происходило написание программы. Была реализована функция `uniquePaths`, отвечающая за алгоритм и представленная на изображении 3.

```

int uniquePaths(int m, int n) {
    std::vector<int> upperrow(n, 1);

    for (int i = 1; i < m; i++) {
        std::vector<int> currentrow(n, 1);
        for (int j = 1; j < n; j++) {
            currentrow[j] = currentrow[j - 1] + upperrow[j];
            upperrow[j] = currentrow[j];
        }
    }
    return upperrow[n - 1];
}

```

Изображение 3 – функция uniquePaths

На вход функция принимает m и n – размеры поля. Далее создаётся массив `upperrow`, состоящий из единиц. В коде он используется для хранения предыдущего ряда таблицы промежуточных результатов.

Затем идёт цикл `for`, который проходит по рядам таблицы, начиная со второго и вплоть до последнего, совершая $m-1$ итераций. Первый ряд пропускается поскольку число способов попасть в каждую его клетку уже учитывается при инициализации `upperrow` и равно 1, так как робот может попасть в клетки первого ряда единственным способом – двигаясь влево. На каждой итерации создаётся вектор `currentrow`, который хранит ряд таблицы, рассматриваемый и формируемый на данной итерации.

Далее идёт вложенный цикл `for`, проходящий столбцы со второго по n -ый, совершая $n-1$ итераций. На данном этапе поэлементно вычисляются промежуточные результаты для текущего ряда (`currentrow`) и происходит поэлементное обновление предыдущего (`upperrow`). То есть, как только результат `currentrow[j]` был вычислен, он тут же записывается в `upperrow[j]`, поскольку изначальное значение `upperrow[j]` больше не требуется для данной итерации.

В итоге, внешний цикл `for` проходит все ряды таблицы и в `upperrow` сохраняется последний ряд промежуточных результатов. Функция же возвращает

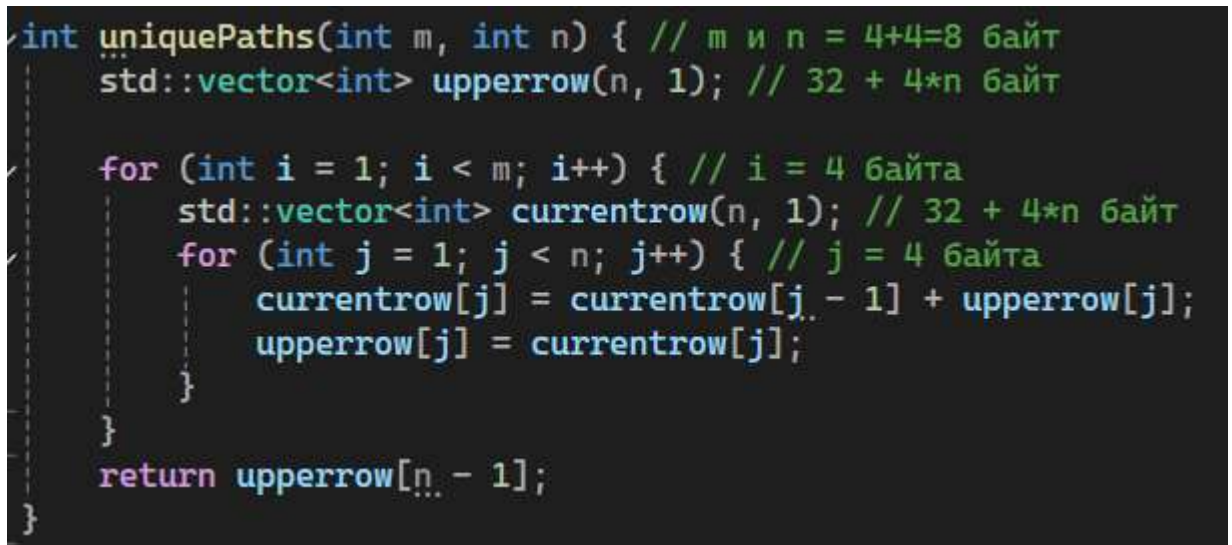
самое правое значение в `upperrow`, то есть количество способов попасть в правую нижнюю клетку.

В конечном счёте, можно сказать, что данная задача не была слишком трудной в реализации.

4. Экспериментальная часть

На данном этапе были произведены подсчет затрачиваемой памяти, без учёта входного массива, и асимптотический анализ написанной программы. Также решение задачи было отправлено на проверку на сайте `leetcode.com`.

4.1. Подсчёт памяти



```
int uniquePaths(int m, int n) { // m и n = 4+4=8 байт
    std::vector<int> upperrow(n, 1); // 32 + 4*n байт

    for (int i = 1; i < m; i++) { // i = 4 байта
        std::vector<int> currentrow(n, 1); // 32 + 4*n байт
        for (int j = 1; j < n; j++) { // j = 4 байта
            currentrow[j] = currentrow[j - 1] + upperrow[j];
            upperrow[j] = currentrow[j];
        }
    }
    return upperrow[n - 1];
}
```

Изображение 4 – подсчёт затрачиваемой памяти

Таким образом, максимальное количество затрачиваемой памяти в моменте будет $8 + 32 + 4 * n + 4 + 32 * 4 * n + 4 = 8 * n + 80$ байт. Соответственно, пространственная сложность алгоритма составляет $O(n)$.

4.2. Подсчёт асимптотики

Поэтапно рассмотрим приблизительную оценку:

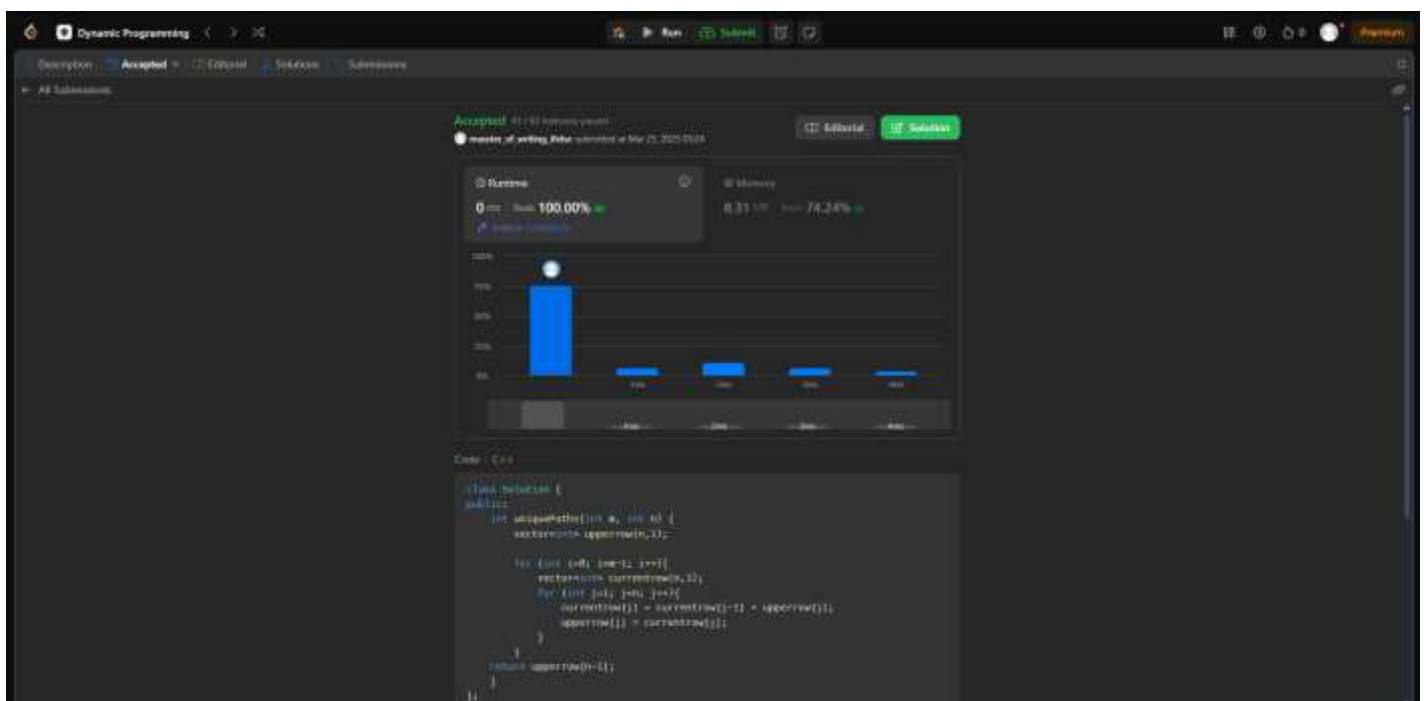
1. Инициализация вектора `upperrow` и заполнение его единицами даёт n итераций
2. Внешний цикл `for` совершает $m-1$ итераций

3. Инициализация вектора `currentrow`, происходящая внутри цикла `for`, даёт n итераций на каждую итерацию внешней петли
4. Вложенный цикл `for` проходит массив со второго по n -ый элемент, что даёт $n-1$ итераций.

В итоге, асимптотическую оценку алгоритма можно свести к $n+(2*n-1)*m-1$, что соответствует временной сложности $O(n*m)$.

4.3. Тесты

На данном этапе программа была отправлена на проверку на сайте `leetcode.com`. В итоге, написанный код успешно прошёл все тесты, что показано на изображении 5.



Изображение 5 – Скриншот, подтверждающий, что все тесты были пройдены успешно

62. Unique Paths

Solved ✓

Medium

Topics

Companies

There is a robot on an $m \times n$ grid. The robot is initially located at the **top-left corner** (i.e., `grid[0][0]`). The robot tries to move to the **bottom-right corner** (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

Given the two integers `m` and `n`, return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Изображение 6 – Скриншот, подтверждающий, что задача была решена

4.4. Анализ метода решения

Анализируя решение с помощью применения динамического программирования, можно сделать вывод, что оно является оптимальным решением для всей задачи, поскольку оно может быть построено из оптимальных решений для предыдущих клеток. То есть, число способов попасть в каждую последующую клетку строится из количества способов попасть в две предыдущие, находящиеся слева и сверху. В динамическом программировании мы сохраняем результаты подзадач, что позволяет избежать лишних вычислений, как например в рекурсивном подходе, когда алгоритм рекурсивно идёт по полю и на каждой итерации рассматривается два возможных направления. В ДП при встрече уже вычисленной промежуточной задачи, нужный результат просто извлекается из списка.

5. Заключение

В ходе выполнения данной лабораторной работы мной была решена задача 62. Unique Paths, доступная на сайте leetcode.com и имеющая уровень «medium». Была написана программа для нахождения числа возможных способов попасть из верхней левой клетки в правую нижнюю и проанализирована её работа.

Программа успешно прошла все тесты на сайте leetcode.com. Анализ результата подтвердил корректность и эффективность метода динамического программирования в данном случае.

6. Приложение

ПРИЛОЖЕНИЕ А

Листинг файла main.cpp

```
class Solution {  
  
public:  
  
    int uniquePaths(int m, int n) {  
  
        vector<int> upperrow(n,1);  
  
  
        for (int i=0; i<m-1; i++){  
  
            vector<int> currentrow(n,1);  
  
            for (int j=1; j<n; j++){  
  
                currentrow[j] = currentrow[j-1] + upperrow[j];  
  
                upperrow[j] = currentrow[j];  
  
            }  
  
        }  
  
        return upperrow[n-1];  
    }  
};
```